
DSC 40B - Quiz 01

Name:

Student ID:

About the quizzes:

- Quizzes in DSC 40B are *optional* and graded pass/fail.
- A score of 70% or higher earns a “pass” and 1.5 credits toward your final grade.
- If you don’t pass, no credits are earned, but it doesn’t hurt your grade.
- You have 30 minutes to complete the quiz.
- Unfortunately, we can’t answer clarifying questions during the quiz. If you think a question has a bug or is unclear, please let us know in a private post on Campuswire after the quiz, and we’ll take it into account when grading.

Problem 1.

What is the time complexity of the following function?

```
def foo(arr):  
    """`arr` is a list containing  $n$  numbers."""  
    x = sum(arr) - min(arr)  
    if max(arr) > 100:  
        return x  
    else:  
        return -x
```

- ☐ $\Theta(1)$ ☐ $\Theta(\log n)$ ☒ $\Theta(n)$ ☐ $\Theta(n \log n)$ ☐ $\Theta(n^2)$ ☐ $\Theta(n^3)$ ☐ $\Theta(n^4)$

Solution: $\Theta(n)$. `sum`, `min`, and `max` each take $\Theta(n)$ time, but each is called only once, and there is no loop. So the total is $\Theta(n) + \Theta(n) + \Theta(n) = \Theta(n)$.

This problem was similar to Problem #001 in the Quiz 01 practice problems.

Problem 2.

What is the time complexity of the following function?

```
def foo(arr):  
    """`arr` is a list containing  $n$  numbers."""  
    for x in arr:  
        if x > sum(arr) / 2:  
            print('big!')
```

- ☐ $\Theta(1)$ ☐ $\Theta(\log n)$ ☐ $\Theta(n)$ ☐ $\Theta(n \log n)$ ☒ $\Theta(n^2)$ ☐ $\Theta(n^3)$ ☐ $\Theta(n^4)$

Solution: $\Theta(n^2)$.

The thing to watch out for here is that `sum(arr)` is called *inside* the loop, and so it is executed n times. Each time, `sum(arr)` takes $\Theta(n)$ time, so the total time is $\Theta(n) \cdot \Theta(n) = \Theta(n^2)$.

This is an example of an inefficient implementation. A better implementation would be to compute `sum(arr)` once, store it in a variable, and then use that variable inside the loop.

This problem was similar to Problem #007 in the Quiz 01 practice problems.

Problem 3.

What is the time complexity of the following function?

```
def foo(n):  
    for i in range(n**2):  
        for j in range(n):  
            for k in range(5):  
                print(i + j + k)
```

- ☐ $\Theta(1)$ ☐ $\Theta(\log n)$ ☐ $\Theta(n)$ ☐ $\Theta(n \log n)$ ☐ $\Theta(n^2)$ ☒ $\Theta(n^3)$ ☐ $\Theta(n^4)$

Solution: $\Theta(n^3)$. The loops are independent, so the body runs $n^2 \cdot n \cdot 5 = 5n^3$ times. The constant 5 doesn't change the growth rate.

This problem was similar to Problem #008 in the Quiz 01 practice problems.

Problem 4.

What is the time complexity of the following function? (Recall that `//` is integer division: `n // 100` divides `n` by 100 and rounds down to the nearest integer.)

```
def foo(n):  
    for i in range(n**2 - 3*n + 50):  
        for j in range(n // 100):  
            print(i + j)
```

☐ $\Theta(1)$ ☐ $\Theta(\log n)$ ☐ $\Theta(n)$ ☐ $\Theta(n \log n)$ ☐ $\Theta(n^2)$ ☒ $\Theta(n^3)$ ☐ $\Theta(n^4)$

Solution: $\Theta(n^3)$. The outer loop runs $n^2 - 3n + 50 = \Theta(n^2)$ times and the inner loop runs $\lfloor n/100 \rfloor = \Theta(n)$ times, so the body runs $\Theta(n^2) \cdot \Theta(n) = \Theta(n^3)$ times.

Or, to put it another way, the innermost loop body runs exactly $(n^2 - 3n + 50) \cdot \lfloor n/100 \rfloor$ times, which is $\Theta(n^3)$.

This problem was similar to Problem #010 in the Quiz 01 practice problems.

Problem 5.

What is the time complexity of the following function?

```
def foo(n):  
    for i in range(n):  
        for j in range(n):  
            print(i + j)  
        for k in range(n**2):  
            print(i + k)
```

☐ $\Theta(1)$ ☐ $\Theta(\log n)$ ☐ $\Theta(n)$ ☐ $\Theta(n \log n)$ ☐ $\Theta(n^2)$ ☒ $\Theta(n^3)$ ☐ $\Theta(n^4)$

Solution: $\Theta(n^3)$. Each iteration of the outer loop runs the two inner loops one after the other, taking $\Theta(n) + \Theta(n^2) = \Theta(n^2)$ time. The outer loop runs n times, for a total of $\Theta(n^3)$.

This problem was similar to Problem #013 in the Quiz 01 practice problems.

Problem 6.

What is the time complexity of the following function?

```
def foo(n):  
    i = n  
    while i > 1:  
        i = i / 3  
        print(i)
```

☐ $\Theta(1)$ ☒ $\Theta(\log n)$ ☐ $\Theta(n)$ ☐ $\Theta(n \log n)$ ☐ $\Theta(n^2)$ ☐ $\Theta(n^3)$ ☐ $\Theta(n^4)$

Solution: $\Theta(\log n)$. After k iterations, $i = n/3^k$. The loop stops once this is at most 1, which takes about $\log_3 n = \Theta(\log n)$ iterations.

This problem was similar to Problem #016 in the Quiz 01 practice problems.

Problem 7.

What is the time complexity of the following function?

```
def foo(n):
    i = 1
    while i < n:
        i = i * 3
        for j in range(n):
            print(i + j)
```

- ☐ $\Theta(1)$ ☐ $\Theta(\log n)$ ☐ $\Theta(n)$ ☒ $\Theta(n \log n)$ ☐ $\Theta(n^2)$ ☐ $\Theta(n^3)$ ☐ $\Theta(n^4)$

Solution: $\Theta(n \log n)$. Since i triples on each iteration, the outer loop runs $\Theta(\log n)$ times. On each of these, the inner loop runs n times, for a total of $\Theta(n \log n)$.

This problem was similar to Problem #019 in the Quiz 01 practice problems.

Problem 8.

What is the time complexity of the following function?

```
def foo(n):
    for i in range(n):
        for j in range(i, n):
            for k in range(10):
                print(i + j + k)
```

- ☐ $\Theta(1)$ ☐ $\Theta(\log n)$ ☐ $\Theta(n)$ ☐ $\Theta(n \log n)$ ☒ $\Theta(n^2)$ ☐ $\Theta(n^3)$ ☐ $\Theta(n^4)$

Solution: $\Theta(n^2)$. The loops are dependent. On the i th outer iteration, the middle loop runs $n - i$ times, so the middle loop runs $n + (n - 1) + \dots + 1 = n(n + 1)/2 = \Theta(n^2)$ times in total. The innermost loop does constant work (10 iterations) each time.

A quicker shortcut is to recognize that the outer two loops are very similar to the dependent nested loop we analyzed when solving the tallest doctor problem in lecture, which took $\Theta(n^2)$ time. There is a third loop here, but it iterates only a constant number of times (10), so we can effectively ignore it.

This problem was similar to Problem #023 in the Quiz 01 practice problems.

Problem 9.

What is the time complexity of the following function?

```
def foo(n):
    for i in range(n**2):
        for j in range(i):
            print(i + j)
```

- ☐ $\Theta(1)$ ☐ $\Theta(\log n)$ ☐ $\Theta(n)$ ☐ $\Theta(n \log n)$ ☐ $\Theta(n^2)$ ☐ $\Theta(n^3)$ ☒ $\Theta(n^4)$

Solution: $\Theta(n^4)$.

The key to this problem is recognizing that what we have here is actually an arithmetic sum, just like we had with the tallest doctor problem in lecture. However, here the outer loop has a bigger range: it runs n^2 times, not n times.

If we go through the same exercise and count how many times the innermost loop body runs on each

outer iteration, we get:

Outer iteration number	1	2	3	...	α	...	$n^2 - 1$	n^2
Number of executions	0	1	2	...	$\alpha - 1$	...	$n^2 - 2$	$n^2 - 1$

We get the total number of executions of the `print` by summing up the second row:

$$\sum_{\alpha=1}^{n^2} (\alpha - 1) = 0 + 1 + 2 + \dots + (n^2 - 2) + (n^2 - 1).$$

This is an arithmetic sum: each term is one more than the one to its left. However, it goes up to $n^2 - 1$ instead of n . Remember that $1 + 2 + 3 + \dots + n = \frac{n(n+1)}{2}$. Replacing n with $n^2 - 1$, we get

$$\frac{(n^2 - 1)n^2}{2} = \Theta(n^4).$$

This problem was similar to Problem #030 in the Quiz 01 practice problems.

Problem 10.

Exactly how many times is `print` called when `foo(n)` is run?

```
def foo(n):
    for i in range(n):
        for j in range(i + 1, n):
            print(i, j)
```

- ☐ n^2
☒ $\frac{n(n-1)}{2}$
☐ $\frac{n(n+1)}{2}$
☐ $\frac{n^2}{4}$
☐ $n(n-1)$

Solution: $\frac{n(n-1)}{2}$.

One way of solving this is to remember that this is more or less the same code for the tallest doctor problem from lecture. There, we calculated that the innermost loop body executes $\frac{n(n-1)}{2}$ times.

If you didn't remember that, you can find it by trying small values of n and looking for a pattern. When $n = 1$, the outer loop runs once, with $i = 0$, and the inner loop is over `range(1, 1)`, which is empty. So there are no prints. When $n = 2$, the $i = 0$ iteration prints once and the $i = 1$ iteration doesn't print, for 1 print in total. Repeating this for several more values of n , we get:

n	1	2	3	4	...	n
Number of prints	0	1	$2 + 1$	$3 + 2 + 1$	...	$(n - 1) + \dots + 1$

In general, the $i = 0$ iteration prints $n - 1$ times, the $i = 1$ iteration prints $n - 2$ times, and so on, so the total is

$$(n - 1) + (n - 2) + \dots + 1 + 0 = \frac{n(n - 1)}{2}.$$

This problem was similar to Problem #025 in the Quiz 01 practice problems.